

차선이탈 자동 복귀 시스템

Lane Departure Auto-Recovery System

조이스틱 수동 주행과 AI 기반 차선이탈 자동 개입을 결합한 SDV 축소 구현

TEAM

서준상 전자전기공학부

정은진 소프트웨어공학부

LDAR

2026

01

학습 진행 현황

Curriculum Progress

02

주제 소개

Background & Scenario

03

차별점

Key Differentiators

04

차선 이탈 감지 파이프라인

Detection Pipeline

05

AI 모델 선정

MobileNet-V2 + U-Net

06

차선 이탈 판정 로직

Decision Logic

07

보드별 역할

AI-G · D3-G · VCP-G

08

통신 구조

Inter-board Communication

09

CAN 메시지 정의

CAN Message Spec

10

제어권 오버라이드

Control Authority

11

필요 물품

Required Materials

12

테스트 환경

Test Environment

학습 진행 현황

튜토리얼 커리큘럼 진척 상황

● 완료 (Completed)

● 현재 지점 (Current)

○ 예정 (Planned)

● D01-D02	환경 구성	Ubuntu/WSL · Yocto · D3-G R5 빌드
● D02-D03	VCP-G 주변장치	GPIO · ADC · I2C · SPI · PDM
● D03-D04	CAN & IPC 통신	D3-G A72↔R5 IPC · VCP-G CAN 수신
● D05	Control Zone	L298N DC 모터 · 서보 · LED 통합
● D06	AI-G 환경 + NPU	tc-nn-toolkit · tc-nn-app · YOLOv8s
● D07	Sensing Zone AI 모델 탑재	UFLD 차선 검출 · 추론 결과 확인
○ D08-D09	HPC Zone · 통합	QT 계기판 App · Ethernet · Panel App

현재 지점 AI-G 보드에 AI 모델(YOLOv8s · UFLD) 탑재 및 NPU 추론 실행까지 완료. 이후 단계는 프로젝트 주제에 맞게 확장 진행 예정.

BACKGROUND

왜 이 주제인가?

상용차에는 이미 **LDWS(차선이탈 경고)**와 **LKAS(차선유지 보조)**가 ADAS 핵심 기능으로 탑재되어 운전자 부주의로 인한 사고를 줄이고 있다.

본 프로젝트는 이 기능을 Telechips Zonal 아키텍처 기반의 모형 차량에 직접 구현하여, 카메라 영상 → NPU 추론 → 차량 제어까지 이어지는 SDV 데이터 흐름을 축소 재현한다.

SCENARIO

어떻게 동작하는가?

- 1 수동 주행**
사용자가 조이스틱으로 모형 차량을 자유롭게 조작
- 2 실시간 차선 인식**
AI-G 카메라가 전방 차선을 감지, 실선/점선을 구분
- 3 이탈 임박 감지**
실선 쪽 접근 시 경고음 + 제어권 강제 이양
- 4 자동 복귀**
보드가 서보 조향을 제어하여 차선 중앙으로 유도
- 5 제어권 반환**
안전 구역에 충분히 머무르면 사용자에게 반환

01

LDWS → LKAS

경고를 넘어선
실제 조향 개입

LDWS(경고)에서 멈추지 않고
LKAS(제어)까지 구현. 서보 조향각을
직접 조정하여 차량을 차선 중앙으로
자동 복귀시킨다.

02

Context-aware

실선 · 점선
조건부 오버라이드

실선은 절대 넘지 않아야 할 경계,
점선은 차선 변경이 허용된 경계.
차선 종류에 따라 개입 조건을
분기하여 과도한 간섭을 방지한다.

03

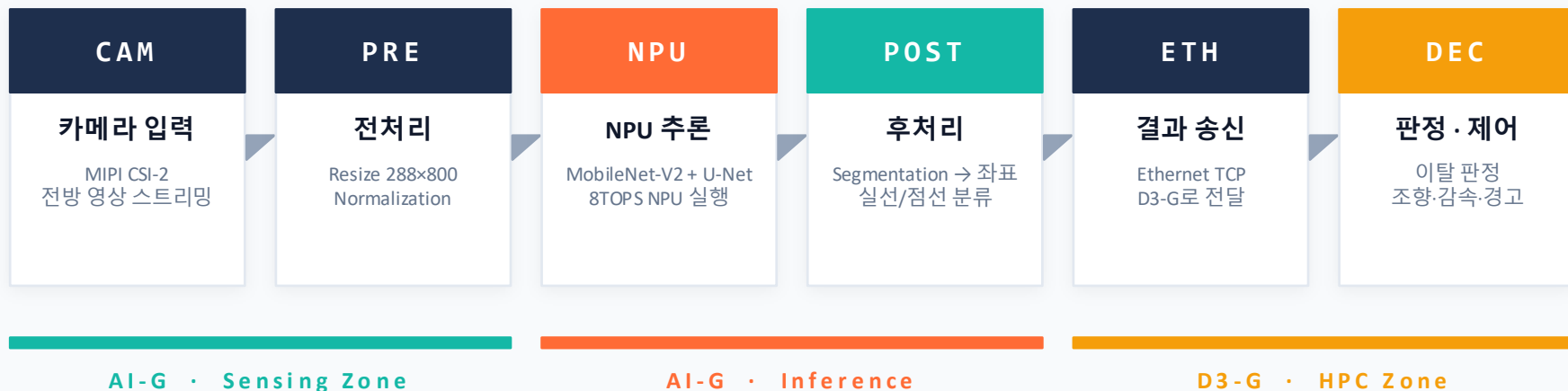
Authority Handoff

사용자 ↔ 자율
제어권 전환 UX

수동 주행 중 위험 상황 감지 시
제어권을 자동 이양하고, 안정 상태
복귀 후 다시 사용자에게 반환하는
명확한 상태 머신을 구현한다.

차선 이탈 감지 파이프라인

Camera → NPU → Detection → Decision → Action



핵심 포인트

- ▶ 추론은 전적으로 AI-G NPU에서 수행 — CPU는 전·후처리와 전송만 담당하여 실시간성 확보
- ▶ 이탈 판정과 제어 로직은 D3-G A72가 전담 — 인지(Sensing)와 제어(Control)의 책임 분리
- ▶ Ethernet(패킷) → IPC(CPU↔MCU) → CAN(MCU↔차량) — Zonal 아키텍처의 계층적 데이터 흐름

선정 모델

MobileNet-V2 + U-Net

Lightweight Semantic Segmentation

BACKBONE

MobileNet-V2

초경량 딥러닝 네트워크. Depthwise Separable Convolution과 Inverted Residual 구조로 파라미터 수와 연산량을 대폭 감소. NPU 리소스가 제한된 AI-G 환경에 적합.

ARCHITECTURE

U-Net

영상 안의 모든 픽셀을 분석해 도로 · 차선 · 배경을 정교하게 구분하는 Semantic Segmentation 모델. Encoder-Decoder + Skip Connection으로 경계 정보 보존에 강함.

WHY THIS COMBO?

선정 이유

- ✓ 차선 곡률을 픽셀 단위로 정밀 인식
- ✓ 실선/점선의 연속성 차이를 포착 가능
- ✓ NPU 친화적 CNN 구조
- ✓ 실시간 추론 가능한 경량 크기

D3-G로 전달할 데이터

- 좌/우 차선 좌표 (픽셀 단위)
- 차선 종류 — 실선(1) / 점선(2)
- 이탈 위험도 (0.0 ~ 1.0)
- 타임스탬프

차선 이탈 판정 로직

D3-G A72에서 수행되는 판정 프로세스

1

입력 데이터 수신

AI-G로부터 좌/우 차선 좌표, 차선 종류

2

차량 중심 기준 거리 계산

프레임 하단 중심점과 좌/우 차선 픽셀 거리

3

접근 속도(rate) 계산

이전 프레임 대비 거리 변화량

4

임계값 비교 + 분기

차선 종류 · 거리 · 접근속도로 상태 결정

DECISION MATRIX

SAFE

거리 > Tout

사용자 제어 유지

WARNING

거리 < Tin & 점선 쪽

경고음만

OVERRIDE

거리 < Tin & 실선 쪽

제어권 이양 + 조향 복귀

CRITICAL

차선 접촉/통과

즉시 감속 + 조향 복귀

보드별 역할 개요

Sensing · HPC · Control — 3개 보드의 책임 분리

SENSING ZONE

AI-G

인지
(Perception)

카메라 영상을 NPU 추론하여 차선
정보 추출

A53 + NPU 8TOPS
2GB LPDDR4X

HPC ZONE

D3-G

판단
(Decision)

사용자 입력과 인지 결과를 통합하
여 제어 명령 생성

A72 (Linux)
+ R5 (FreeRTOS)

CONTROL ZONE

VCP-G

구동
(Actuation)

CAN 메시지 수신 후 모터 · LED · 경고
음 제어

MCU
+ FreeRTOS

데이터 흐름: Ethernet → IPC (A72 ↔ R5) → CAN

AI-G · Sensing Zone

인지(Perception) — 카메라 영상 기반 차선 인식

BOARD

AI-G

CPU Cortex-A53 Quad

NPU 8 TOPS (1.0GHz)

Memory 2GB LPDDR4X

Camera MIPI CSI-2

Network Gigabit Ethernet

OS Linux

RESPONSIBILITIES

AI-G가 담당하는 작업

1 카메라 입력 캡처

MIPI CSI-2 연결 카메라로부터 전방 주행 영상을 실시간 획득

2 NPU 기반 차선 추론

tc-nn-app 런타임 상에서 MobileNet-V2 + U-Net 모델을 8TOPS NPU로 실행

3 후처리: 좌표화 + 실선/점선 분류

Segmentation 결과를 차선 좌표 배열과 차선 종류 레이블로 변환

4 결과 데이터 터미널 출력 및 송신

추론 결과를 터미널에서 로그로 확인(디버그) 후 Ethernet을 통해 D3-G로 TCP 전송

D3-G · HPC Zone

판단(Decision) — A72와 R5의 역할 분리

D3-G (TCC8050)

듀얼 코어 구조: Application Core (Cortex-A72) + MCU Core (Cortex-R5)

Cortex-A72

Application Core · Linux

로직 · 판정 · UI

- **사용자 입력 처리**
조이스틱(USB HID) 입력값을 조향/속도로 해석
- **차선 데이터 수신**
AI-G로부터 Ethernet TCP로 인지 결과 수신
- **이탈 판정 및 제어 결정**
거리·접근속도·실선여부 조합으로 오버라이드 판단
- **제어 명령을 R5에 IPC 전달**
최종 조향·속도·LED·경고음 명령을 IPC 패킷으로 전송
- **계기판 App UI 출력**
QT 기반 앱으로 제어권 상태, 속도 등 시각화

Cortex-R5

MCU Core · FreeRTOS

IPC · CAN 송신

- **IPC 수신**
A72로부터 제어 명령 패킷 수신 (헤더 + 명령 + CRC)
- **CAN 드라이버 제어**
지정된 CAN ID와 데이터로 프레임 생성
- **VCP-G로 CAN 송신**
CAN Transceiver를 통해 VCP-G의 R5로 메시지 전달
- **안전 영역 분리**
A72 장애 시에도 차량 제어 경로를 독립적으로 유지

BOARD

VCP-G

Core	MCU
OS	FreeRTOS
CAN	Receiver
Interfaces	GPIO, PDM,
	I2C
Drives	DC Motor,
	Servo, LED

ACTUATION OUTPUTS

무엇을 제어하는가?

DC 모터 (후륜 구동)

L298N IN1/IN2로 회전 방향, PDM 듀티(0~80%)로 속도 제어

CAN 0x106

서보 모터 (전륜 조향)

50Hz PDM · 듀티 0.5~2.5ms로 0~127° 각도 출력

CAN 0x107

LED (상태 표시)

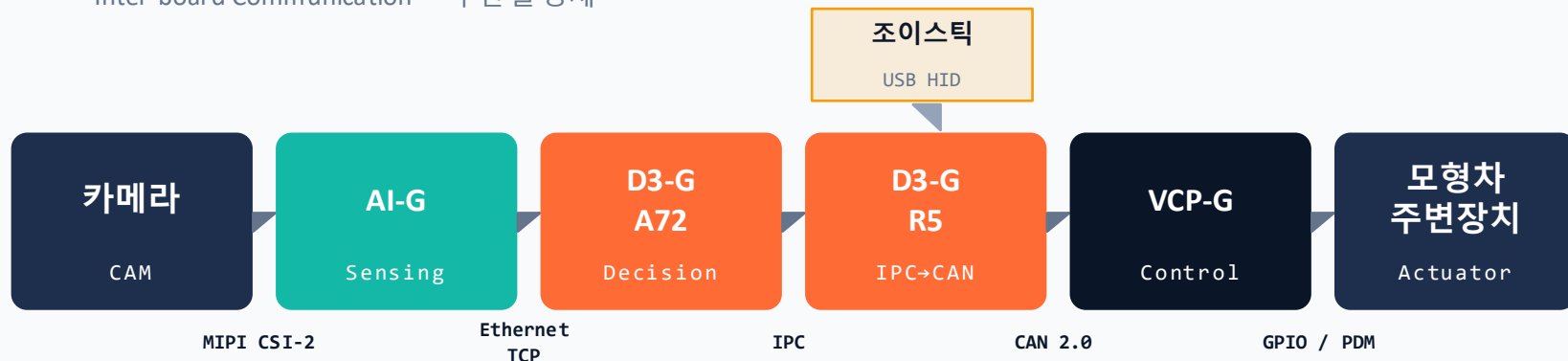
브레이크등, 좌/우 방향지시등, 전조등, 경고등 GPIO ON/OFF

CAN 0x101~0x104

버저 (경고음)

이탈 임박 및 오버라이드 상태에서 활성화, GPIO 토글

CAN 0x103



구간별 통신 상세

구간	규격	주요 데이터
카메라 → AI-G	MIPI CSI-2	원본 영상 프레임 (RAW)
AI-G → D3-G(A72)	Ethernet / TCP	좌/우 차선 좌표, 실선·점선 라벨, 위험도, 타임스탬프
조이스틱 → D3-G(A72)	USB HID	조향 축(X), 속도 축(Y), 트리거 버튼
D3-G A72 ↔ R5	IPC (/dev/tcc_ipc_micom)	제어 명령 패킷: 헤더 + 명령 + CRC
D3-G(R5) → VCP-G	Classical CAN 2.0 (11-bit ID)	ID별 제어 메시지 (LED, 모터, 경고)
VCP-G → 모형차	GPIO · PDM(PWM) · I2C	방향 핀, PWM 듀티, LCD

CAN 메시지 정의

기존 can_vcp_ctrl.c 그대로 사용 + 신규 2개 추가

메시지	CAN ID	Data 구조	구분
Brake Light	0x101	[0]: on=0x01 / off=0x02	기존
Turn Signal	0x102	[0]: left=0x01, right=0x02 [1]: on/off	기존
Emergency Signal	0x103	[0]: on=0x01 (경고음 연동)	기존
Head Light	0x104	[0]: on=0x01 / off=0x02	기존
Vehicle Speed	0x106	[0]: 0~80 (DC 모터 duty %)	기존
Wheel Angle	0x107	[0]: 0~127 (서보 각도)	기존
Control Authority	0x110	[0]: USER=0x01 / BOARD=0x02	신규
Lane Status	0x111	[0]: safe/warn/depart [1]: solid/dashed	신규

신규 메시지 설계

0x110 Control Authority

현재 제어권 주체를 표시.
계기판 App UI 색상·아이콘,
조향 입력 무시 여부를 결정
하는 근거.

0x111 Lane Status

차선 상태와 종류. 계기판에
서 실선/점선 경고를 다르
게 표시하고, 경고음 패턴을
분기하는 데 사용.

⚠ AI-G → D3-G 패킷 포맷 모델 추론 결과의 실제 출력 텐서 구조(차선 포인트 수 · 종류 표현 방식)를 확인한 뒤 확정. 현재 설계 단계.

차량 조작 및 제어권 오버라이드

사용자 ↔ 보드 제어권 전환 상태 머신



오버라이드 진입 조건 (AND)

- ① 한쪽 차선까지의 픽셀 거리 < 진입 임계값 T_{in}
- ② 해당 차선이 실선으로 분류됨
- ③ 차선 쪽으로의 접근속도 > 0 (이탈 진행 중)
- ④ 해당 방향 방향지시등이 켜져 있지 않음

오버라이드 해제 조건 (AND)

- ① 차선까지의 거리 > 해제 임계값 T_{out} (> T_{in} , 채터링 방지)
- ② 위 상태가 홀드시간 T_h (예: 1초) 이상 유지
- ③ 차량 heading이 차선과 평행에 가까움
(각도 편차 < 임계각)

필요한 물품

Required Materials — 보유 / 구매 필요 구분

BOARDS

보드 및 임베디드 장비

● TOPST AI-G 보드	보유	● MIPI CSI 카메라	보유
● TOPST D3-G 보드	보유	● Ethernet 허브·케이블	보유
● TOPST VCP-G 보드	보유	● USB-to-TTL UART	보유

VEHICLE

모형 차량 및 액추에이터

● 모형 차량 새시	보유	● L298N 모터 드라이버	보유
● DC 모터 (후륜 구동)	보유	● LED 다수 (상태 표시)	보유
● 서보 모터 (전륜 조향)	보유	● 점퍼선·저항	보유

INPUT

입력장치·부가 부품

● 조이스틱 (USB HID)	구매	● 모터 전용 배터리 팩	보유/확인
● 피에조 스피커	구매		

ENV

테스트 환경·소프트웨어

● 실선&점선 차선 트랙	제작	● Yocto / FreeRTOS 환경	보유
● tc-nn-toolkit	보유	● 차선 학습 데이터셋	수집/선정
● tc-nn-app			

TRACK

차선 트랙 구성

실선 (Solid) → 절대 넘지 말 것, 오버라이드 발동



점선 (Dashed) → 차선 변경 허용, 경고만

직선+ 완만한 곡선. 실선과 점선 구간을 모두 포함한 경로로 제작

TEST PLAN

검증 시나리오

1 정상 주행

차선 중앙 유지 시 제어권이 사용자에게 있음을 확인

2 실선 이탈

실선 접근 시 경고음 + 조향 복귀 동작 확인

3 점선 이탈 (무방향지시)

경고음만 울리고 오버라이드는 발동 안 함

4 점선 이탈 (방향지시 있음)

경고 없이 차선 변경 허용

5 제어권 복귀

Tout 충족 후 홀드시간 경과 시 사용자에게 반환

시연 환경: 실제 모형을 테스트 트랙에서 주행시키며 카메라 → NPU → 제어 전체 흐름을 실시간 검증

주차별 실행 계획

9주차 ~ 14주차 · 총 6주 로드맵

1

9-10주차

AI-G 모델 탑재 및 검증

SENSING

MILESTONE

차선 인식 좌표 출력 확인

MobileNet-V2 + U-Net 모델 변환 · 양자화 · NPU 컴파일 · AI-G 보드에 모델 배포 후 실시간 추론 실행 · 실선 · 점선 인식 결과 확인 (좌표 맵 출력)

2

11주차

주행 시스템 및 이탈 감지 구현

CONTROL

MILESTONE

이탈 감지 → 자동 복귀 단독 동작

조이스틱 기반 수동 주행 시스템 구현 · 좌표 맵 기반 차선 이탈 감지 알고리즘 구현 · 이탈 시 자동 복원(조향 복귀) 알고리즘 구현

3

12주차

시스템 통합

INTEGRATION

MILESTONE

End-to-End 전체 흐름 동작

AI-G ↔ D3-G ↔ VCP-G 전체 보드 연동 · Ethernet · IPC · CAN 전체 시퀀스 수행 · 제어권 전환 상태 머신 통합 검증

4

13주차

디버깅 및 최적화

DEBUG

MILESTONE

안정성 확보 · 파라미터 확정

엣지 케이스 디버깅 (곡선 구간 · 조명 변화) · 임계값 튜닝 (Tin · Tout · 홀드시간) · 추론 속도 · 응답 지연 최적화

5

14주차

데모 시연 및 촬영

DEMO

MILESTONE

최종 시연 · 영상 제출

테스트 트랙에서 최종 데모 시연 · 시연 영상 촬영 · 발표 자료 정리

Q & A

경청해 주셔서 감사합니다.

Thank you for your attention